Benchmark Results: synsc-context vs Context7 vs Nia

Version: 1.0 | **Date**: 2026-03-14 | **Judge**: Claude Sonnet 4.6 (Anthropic)

Phase Overview

Phase	Task	Method	Queries	Engines Tested
1	Custom retrieval quality	Hand-crafted queries against FastAPI/Pydantic/httpx	55	synsc, Nia, Context7
2	Industry-standard IR	CoSQA + CodeSearchNet, content/file matching	997	synsc, Nia
3	LLM-as-Judge (3D)	Blind Claude Sonnet 4.6, 3 dimensions (rel/comp/spec)	997	synsc, Context7, Nia
4	Enhanced Judge (4D, debiased)	Position-debiased, 4 dimensions + faithfulness, RAGAS metrics	997	synsc, Context7

Why 4 phases? Each phase was designed to address fairness concerns from the prior: - Phase 1 used hand-crafted queries → potential bias toward synsc's design - Phase 2 used industry-standard datasets → but content-matching penalizes engines that transform text - Phase 3 used blind LLM judge → but single-pass scoring has ~10% positional bias - Phase 4 added position debiasing + faithfulness + judge consistency metrics → most rigorous

Full Cross-Engine Results

Phase 1 — Custom Benchmarks (synsc vs Nia vs Context7)

Corpus: FastAPI, Pydantic, httpx (indexed on synsc/Nia; pre-crawled on Context7)

Benchmark (metric)	synsc-context	Context7	Nia	Winner
Retrieval (MRR)	**0.950**	0.267	0.175	synsc (3.6x vs ctx7)
Retrieval (P@1)	**0.900**	0.200	0.100	synsc (4.5x vs ctx7)
Retrieval (P@5)	**0.720**	0.245	0.160	synsc (2.9x vs ctx7)
Retrieval (NDCG@10)	**0.890**	0.306	0.262	synsc (2.9x vs ctx7)
Multi-Hop (Coverage)	**0.967**	0.850	0.867	synsc
Multi-Hop (MRR)	**0.913**	0.690	0.616	synsc (1.3x vs ctx7)
Code QA (Accuracy)	**1.000**	0.200	0.333	synsc (5x vs ctx7)

Benchmark (metric)	synsc-context	Context7	Nia	Winner
Code QA (MRR)	**1.000**	0.167	0.333	synsc (6x vs ctx7)
Adversarial (Accuracy)	**0.800**	0.000	0.000	synsc
Adversarial (Discrimination)	**0.550**	0.000	0.000	synsc
Hallucination Rate	**0%**	22.2%	N/A	synsc
Avg Latency	3,713ms	**2,873ms**	10,017ms	ctx7 (1.3x vs synsc)

Phase 2 — Industry-Standard Validated Retrieval

Measured by content similarity matching (Jaccard + SequenceMatcher) and file-path matching. Each query has a known ground-truth code snippet from the benchmark corpus repo. The engine must return text that closely matches that exact snippet to score a hit.

Why Context7 is excluded: The benchmark corpus repos (`benchmark-corpus-cosqa`, `benchmark-corpus-codesearchnet`) contain specific Python files with known answers. synsc indexes those files directly and returns chunks from them, so retrieved content can be matched against ground truth. Context7 doesn't have these repos — it returns documentation from pre-crawled libraries instead, which can't be content-matched against the corpus. This is a limitation of the metric, not the engine. The LLM judge (Phases 3-4) bypasses this by evaluating usefulness regardless of source.

Dataset (metric)	synsc-context	Nia	Context7	Winner
CoSQA MRR (500q)	**0.629**	0.004	—	synsc (157x)
CoSQA P@1	**0.612**	0.004	—	synsc (153x)
CoSQA Recall@10	**0.656**	0.004	—	synsc (164x)
CoSQA NDCG@10	**0.634**	0.004	—	synsc (159x)
CodeSearchNet MRR (497q)	**0.940**	—	—	synsc
CodeSearchNet P@1	**0.938**	—	—	synsc
CodeSearchNet Recall@10	**0.948**	—	—	synsc
CodeSearchNet NDCG@10	**0.942**	—	—	synsc
CoSQA Avg Latency	**4,036ms**	7,686ms	—	synsc (1.9x)
CodeSearchNet Avg Latency	4,205ms	—	—	—

Phase 3 — LLM-as-Judge (3D, single-pass)

Blind scoring: relevance + completeness + specificity (0-3 each). No position debiasing.

Dataset (metric)	synsc-context	Context7	Nia	Winner
CodeSearchNet				

Dataset (metric)	synsc-context	Context7	Nia	Winner
Relevance (0-3)	**2.861**	0.612	—	synsc (4.7x)
Completeness (0-3)	**2.813**	0.318	—	synsc (8.8x)
Specificity (0-3)	**2.382**	0.330	—	synsc (7.2x)
Total (0-3)	**2.685**	0.420	—	**synsc (6.4x)**
CoSQA				
Relevance (0-3)	1.522	**1.762**	—	ctx7
Completeness (0-3)	1.032	**1.402**	—	ctx7
Specificity (0-3)	0.988	**1.330**	—	ctx7
Total (0-3)	1.181	**1.498**	0.267	**ctx7 (1.27x)**
CoSQA (synsc vs Nia, 50q)				
Total (0-3)	**1.067**	—	0.267	synsc (4x)
Wins	**39 (78%)**	—	3 (6%)	synsc

Phase 4 — Enhanced Judge (4D, position-debiased)

Each query evaluated twice (original + reversed chunk order), scores averaged. Scoring: relevance + completeness + specificity + **faithfulness** (0-3 each).

CodeSearchNet (497 queries, debiased)

Metric	synsc-context	Context7	Ratio
Relevance (0-3)	**1.980**	0.626	3.2x
Completeness (0-3)	**1.877**	0.296	6.3x
Specificity (0-3)	**1.360**	0.402	3.4x
Faithfulness (0-3)	**2.044**	1.296	1.6x
Total (0-3)	**1.815**	0.655	**2.8x**
Wins	**436**	36	**12:1**
Ties	25	25	—

CoSQA (500 queries, debiased)

Metric	synsc-context	Context7	Ratio
Relevance (0-3)	1.164	**1.762**	0.66x
Completeness (0-3)	0.552	**1.352**	0.41x
Specificity (0-3)	0.680	**1.412**	0.48x

Metric	synsc-context	Context7	Ratio
Faithfulness (0-3)	1.596	**2.130**	0.75x
Total (0-3)	0.998	**1.664**	**0.60x**
Wins	109	**341**	**1:3.1**
Ties	50	50	—

Context Quality Metrics (RAGAS-inspired, no LLM)

Metric	CSN synsc	CSN ctx7	CoSQA synsc	CoSQA ctx7
Context Precision	**0.481**	0.191	**0.491**	0.235
Context Density	**0.101**	0.058	0.030	**0.034**
Signal-to-Noise	**0.595**	0.389	**0.643**	0.517
Chunk Diversity	**0.945**	0.925	0.927	**0.938**

Judge Self-Consistency

Metric	CodeSearchNet	CoSQA
Position Consistency	0.553	0.785
Cohen's κ	0.290 (fair)	0.537 (moderate)
Avg Score Drift	1.089	0.382

Debiased vs Non-Debiased

Dataset (engine)	Non-Debiased	Debiased	Drift
CodeSearchNet (synsc)	2.785	1.815	-0.970
CodeSearchNet (ctx7)	0.760	0.655	-0.105
CoSQA (synsc)	1.265	0.998	-0.267
CoSQA (ctx7)	1.680	1.664	-0.016

Enrichment Impact (synsc-context only, basic judge)

Dataset	v2.0 (pre-embedding only)	v3.0 (+ post-retrieval)	Delta
CodeSearchNet Total	2.640	**2.685**	+0.045
CoSQA Total	1.167	**1.181**	+0.014
CodeSearchNet Specificity	2.300	**2.382**	+0.082

Latency Summary

Benchmark	synsc-context	Context7	Nia	Fastest
Custom Retrieval	3,713ms	**2,973ms**	10,017ms	ctx7 (1.3x)
Multi-Hop	3,575ms	**2,764ms**	7,025ms	ctx7 (1.3x)
Code QA	3,549ms	**2,908ms**	7,137ms	ctx7 (1.2x)
Adversarial	3,851ms	**2,846ms**	7,233ms	ctx7 (1.4x)
Hallucination	3,713ms	**2,873ms**	10,017ms	ctx7 (1.3x)
CoSQA (Validated)	**4,036ms**	—	7,686ms	synsc (1.9x)
CodeSearchNet (Judge)	5,118ms	**3,107ms**	—	ctx7 (1.6x)
CoSQA (Judge)	4,806ms	**3,158ms**	—	ctx7 (1.5x)

Note: synsc-context benchmarks ran locally with ~1.1s geographic latency to Supabase. Production latency on Render is significantly lower (co-located).

Missing Results

Gap	Engine	Reason
Phase 2 (Validated IR)	Context7	Content-matching requires indexed corpus files; ctx7 returns docs (metric limitation, not engine limitation)
Phase 2 (CodeSearchNet)	Nia	Not run — see below
Phase 3 (CSN Judge)	Nia	Not run — see below
Phase 4 (Enhanced)	Nia	Not run — see below

Why Nia results were not completed: Filling all gaps requires ~2,000 Nia API credits (497 + 497 + 997 queries, 1 credit per query). Given Nia's existing results — 0.004 MRR on CoSQA (Phase 2) and 0.267 total on the basic LLM judge (Phase 3) — the additional credit spend was not justified. Nia's `repositories` filter does not scope results to a specific repo, which is the root cause of its low scores across all benchmarks. Completing these runs is unlikely to change the competitive picture.

Ecological Validity Warning

CoSQA queries ("how to parse json in python", "python read csv file") are **not representative of real agent usage**. An LLM already knows these answers from training data and would never invoke a context engine for them.

Real agent query (needs retrieval)	CoSQA query (doesn't need retrieval)
"Find where rate limiting is enforced in this repo"	"python check file is readonly"
"What's the schema for chunk_embeddings table?"	"how to parse json from string in python"
"How does the SSE streaming handler work?"	"python convert datetime to unix timestamp"

CodeSearchNet (docstring → function) is closer to real agent use. **CoSQA scores should be weighted lower** when evaluating context engines for agent workflows. Context7's CoSQA advantage (1.67x) applies to a use case where retrieval is unnecessary. synsc-context's CodeSearchNet advantage (2.8x) applies to the use case where retrieval is essential.
